

Multi-Hospital Post-Discharge Outreach Platform

Autonomous AI-Powered Patient Follow-Up, Clinical Triage & Hospital Outreach Operations Platform

Product Requirements Document — Version 2.0

Status: Product Definition & Full Stack AI Engineering Challenge

Target: 3–4 Day Prototype

Role: Full Stack AI Engineer / AI Systems Engineer

1. Project Overview

Hospitals discharge large numbers of patients every day, but post-discharge follow-up is difficult to manage consistently. Staff need to determine which patients require outreach, decide who should be contacted first, make calls within appropriate clinical windows, understand what happened during each interaction, identify patients who may need human attention, and maintain an accurate operational record.

Build a **multi-tenant AI-powered healthcare operations platform** that automates and coordinates this post-discharge outreach process.

The platform should accept patient discharge data from multiple hospitals, create outreach campaigns, determine eligible patients, prioritize them through a capacity-aware outbound queue, conduct AI-assisted voice conversations, collect structured patient information, perform clinical triage using hospital-approved protocols, determine whether human escalation is required, document the interaction, update a mock EHR, notify appropriate staff, and provide operational dashboards and analytics.

The system should feel like a **hospital outreach operations platform**, not simply an AI phone agent.

The central operational questions are:

1. Which patients need follow-up?
2. Who should be contacted first?

3. What happened during outreach?
4. Which patients require human attention?
5. How well is the campaign performing?

The prototype should demonstrate the complete journey from hospital configuration and discharge data through outreach, queue management, AI reasoning, escalation, documentation, and operational reporting.

2. Product Goal and Engineering Principles

The primary goal is to demonstrate that you can build a production-oriented AI system around a healthcare workflow rather than simply integrate an LLM with a calling API.

Patient safety is the highest priority. The system should behave conservatively when information is incomplete or ambiguous. When there is meaningful uncertainty, the safer behavior is to escalate rather than silently classify a potentially concerning patient as routine.

AI decisions must be grounded in available patient information, hospital-approved protocols, retrieved knowledge, and evidence from the conversation. The system must not invent clinical information or provide unsupported medical advice.

Automation must remain controlled. AI must not directly modify database records, diagnose patients, prescribe or change medication, override hospital rules, suppress escalations, or perform arbitrary clinical operations. AI actions must go through validated and authorized application interfaces.

The platform must also be **queue-aware and operationally observable**. Outbound calling has limited capacity, clinical deadlines, retries, patient preferences, and competing campaigns. The system therefore needs an explicit scheduling strategy rather than simply placing every patient into a generic job queue.

Finally, every hospital must remain isolated from every other hospital. Tenant context must be enforced throughout APIs, databases, workers, AI retrieval, tools, background jobs, analytics, and integrations.

3. Users, Roles and Multi-Tenancy

The platform is multi-tenant. A hospital is a logical tenant containing its users, patients, encounters, discharge records, campaigns, protocols, knowledge resources, calls, escalations, notifications, analytics, and configuration.

Tenant isolation must be enforced on the backend and data-access layer rather than relying on frontend filtering. Every patient-related operation should carry explicit tenant context. Hospital A must never be able to retrieve Hospital B's patients, calls, campaigns, protocols, users, or analytics. Background workers and AI retrieval must preserve the same tenant boundary.

The platform should support four primary roles.

Platform Admin manages hospital tenants and platform-level configuration and can monitor aggregate system health, usage, failures, and AI metrics. Access to clinical content should still follow authorization rules rather than automatically granting unrestricted access to every patient's information.

Hospital Admin manages hospital users, configuration, discharge data, protocols, knowledge resources, escalation contacts, and hospital-level analytics.

Campaign Manager creates and operates campaigns, monitors queue progress, starts or pauses outreach, reviews retries and failures, adjusts permitted operational settings, and monitors campaign performance.

Clinical Reviewer reviews escalations, patient summaries, observations, triage evidence, and AI assessments and records the human resolution or follow-up action.

Authentication, authorization, RBAC, and tenant isolation are required parts of the system rather than optional UI features.

4. Hospital Onboarding and Configuration

Platform administrators should be able to create and configure hospitals. Hospital configuration should include the hospital identity and contact information, timezone, permitted calling hours, outreach protocols, escalation contacts, outbound calling capacity, retry rules, notification preferences, knowledge resources, and mock EHR integration settings.

A simple hospital lifecycle is sufficient: a hospital is created, configured, assigned administrators, provided with protocols and operating rules, and then marked ready for campaigns.

Calling windows and clinical follow-up windows should be configurable because different campaigns and hospitals may have different operational requirements.

5. Patient and Discharge Data

The prototype may use a simulated discharge feed containing approximately **200–500 patients across multiple hospitals**. The dataset should contain enough variation to demonstrate queue prioritization, clinical risk, different discharge times, different follow-up requirements, successful contacts, failed attempts, callbacks, and escalations.

At minimum, patient records should contain a patient identifier, encounter information, hospital, discharge timestamp, contact information, care setting, conditions, discharge instructions, follow-up window, risk indicators, relevant medication or care-plan information, and communication preferences.

Use a structured healthcare-oriented representation. A simplified FHIR-like model is acceptable; exact FHIR compliance is not required. The implementation should demonstrate awareness of healthcare entities such as Patient, Encounter, Observation, Condition, Medication, CarePlan, Procedure, Organization, Communication, and Task.

The important requirement is that the application has a meaningful healthcare data model rather than treating a patient as a flat JSON object.

6. EHR Integration and Data Boundaries

The platform must include an EHR abstraction layer. A real EHR is not required for the prototype; a mock EHR is expected.

The interface should support retrieving patient, encounter, discharge, condition, observation, and care-plan information and writing appropriate outreach results such as communication records, structured observations, follow-up tasks, escalation records, or mock encounter-related updates.

AI agents must never directly manipulate database tables. Instead, AI should request a structured operation through a controlled tool, after which the backend performs authorization, schema validation, business-rule validation, and the actual healthcare operation.

The EHR interface should be replaceable so that a mock implementation can later be substituted with a real integration. External operations must be observable and failures must be represented explicitly.

7. Clinical Protocols and Knowledge Retrieval

Each hospital must be able to maintain hospital-specific post-discharge protocols and knowledge. These resources may contain follow-up questions, red-flag indicators, specialty-specific instructions, approved patient guidance, contact windows, escalation contacts, and operational rules.

Retrieval must always be tenant-aware. A patient from Hospital A must never cause the AI system to retrieve Hospital B's protocol or knowledge.

Clinical guidance should preserve its source or protocol reference so that reviewers can understand where a recommendation originated. The AI should retrieve only the context necessary for the current task rather than indiscriminately loading the patient's entire history or every hospital document.

The candidate should explain the retrieval approach and how grounding is enforced.

8. Campaign Management

A campaign represents a group of patients requiring a particular type of post-discharge outreach.

Campaign configuration should include the hospital, campaign name and description, eligibility criteria, clinical follow-up window, calling hours, priority rules, retry limits, campaign dates, outbound capacity, and escalation configuration.

Campaigns should support a lifecycle such as:

State	Meaning
DRAFT	Being configured
READY	Validated and ready to run
SCHEDULED	Waiting for its configured start
RUNNING	Outreach is active
PAUSED	New calls are temporarily stopped
COMPLETED	Campaign work is finished
CANCELLED / FAILED	Optional terminal states

Before activation, the system should show an estimate of the workload, such as the number of eligible patients and expected outreach attempts.

Pausing a campaign must stop new outbound work while allowing active calls to finish safely. Resuming should recalculate eligible work rather than blindly restarting every previous task.

Campaign managers should be able to start, pause, resume, cancel where appropriate, and reprioritize campaigns or permitted operational settings.

9. Patient Eligibility

Eligibility determines which patients should enter a campaign.

The system should consider the hospital, discharge status and date, campaign rules, clinical follow-up window, communication eligibility or consent assumptions, patient-specific requirements, and existing outreach status.

Eligibility should be evaluated consistently and explainably. Operational errors must not silently remove patients from outreach. If eligibility processing fails, the system should expose the failure and provide a recovery path.

10. Outbound Queue and Scheduling

The outbound queue is one of the **most important grading areas** of this project.

Assume, for example, that a campaign contains 300 eligible patients while the hospital has capacity for only 10 concurrent outbound calls. The platform must centrally enforce that limit even when multiple workers attempt to schedule calls simultaneously.

The queue should optimize successful outreach within the patient's clinical window rather than simply processing patients in insertion order. Scheduling should consider factors such as clinical risk, time since discharge, time remaining before the follow-up deadline, patient availability, previous failed attempts, requested callback times, campaign priority, and retry state.

You must design and document an actual prioritization algorithm. Do not simply state that the queue uses "priority." Explain how priority is calculated, how competing campaigns are handled, how starvation is prevented, and why one patient is selected before another.

The scheduler should also account for deadline pressure. A patient who has a high-priority campaign but many hours remaining may reasonably be scheduled after a patient whose clinical follow-up window is about to expire.

The queue must safely coordinate reservation of capacity. Two workers must not accidentally reserve the same slot or exceed the configured concurrency limit.

11. Queue States, Retries, Callbacks and Failure Handling

Each outreach task should have an explicit state. A reasonable model includes:

PENDING, SCHEDULED, CALLING, CONNECTED, COMPLETED, NO_ANSWER, BUSY, VOICEMAIL, DROPPED, RETRY_SCHEDULED, CALLBACK_SCHEDULED, ESCALATED, MANUAL_FOLLOW_UP, and FAILED.

You may adapt these states, but the distinction between different outcomes should remain clear.

The system should distinguish a busy line, voicemail, no answer, invalid number, rejected call, network failure, dropped call, successful completion, and technical failure because these outcomes influence future scheduling.

Retry behavior must consider attempt count, clinical window, calling hours, patient preference, campaign priority, and previous attempt timing. Retries should use sensible backoff rather than repeatedly calling a patient immediately.

A dropped call must persist in its partial state. The next attempt should retain relevant context instead of necessarily restarting the interaction from zero.

A patient who requests a callback should have the requested time captured and scheduled explicitly. The patient should not simply be returned to the generic queue.

After the maximum number of retries, the system should create a visible manual follow-up task and notify the appropriate staff.

Calling must respect hospital timezone, campaign calling windows, and patient communication preferences where available.

Near the end of a clinical window, the scheduler should prioritize patients who are at risk of missing the deadline.

Workers must also recover from failures. If a worker crashes while holding capacity, the system should detect the stale task, release the capacity, and either retry or move the task to an explicit manual/failed state.

The implementation must prevent duplicate calls and duplicate side effects.

12. Required Queue Simulation

A queue simulation is mandatory because the evaluator must be able to observe the scheduling system without depending entirely on real telephony.

Create a simulation containing approximately **20–30 mock patients** with mixed clinical risk, different deadlines, limited calling capacity, successful calls, no answers, busy outcomes, dropped calls, retries, callbacks, and escalations.

The evaluator should be able to see the queue change dynamically and verify that concurrency limits are respected.

You must submit a short queue design document explaining the state machine, priority calculation, retry and backoff strategy, concurrency control, callback handling, deadline behavior, fairness/starvation prevention, failure recovery, and manual escalation behavior.

The design must justify the chosen approach rather than merely describing implementation details.

13. AI Agent Architecture

Do not implement the entire product as one unrestricted AI agent.

The system should separate AI responsibilities into logical components. At minimum, the design should distinguish the **Voice Intake Agent**, **Clinical Triage Agent**, **Escalation Decision System**, and **Documentation Agent**.

The Voice Intake Agent manages the patient conversation. It introduces the outreach, verifies that the interaction can proceed, asks protocol-driven questions, captures patient responses, identifies symptoms or concerns, handles clarification, and determines when the interaction should end or be transferred.

The Clinical Triage Agent converts conversation information into structured observations and compares those observations against approved hospital protocols.

The Escalation Decision System combines independent assessments and determines whether the case requires human attention.

The Documentation Agent converts the interaction into a structured record containing the call summary, patient-reported symptoms, observations, outcome, triage result, escalation status, and follow-up requirements.

The exact model providers and agent framework are intentionally left open.

14. Voice Intake and Conversation

The voice interaction should behave like a structured healthcare outreach conversation rather than an unrestricted chatbot.

The agent should use patient and campaign context, follow the hospital-approved protocol, ask the necessary questions, identify relevant symptoms, handle reasonable conversational variation, and avoid making unsupported medical claims.

The conversation should collect enough structured information for downstream triage and documentation.

If real outbound calling is implemented, it is a strong enhancement. However, a deterministic call simulator is acceptable for demonstrating the core queue and AI workflow.

The system should maintain a call record containing the patient, campaign, timestamps, duration, outcome, conversation record or transcript where available, AI outputs, triage result, escalation result, and documentation status.

15. Clinical Triage and Structured AI Outputs

Whenever application logic depends on AI, the AI must produce structured output that can be validated by the backend.

A triage result should contain a classification such as **routine, concerning, urgent, or uncertain**, together with observed indicators, evidence from the conversation, relevant protocol references, confidence or uncertainty information, and an escalation recommendation.

The exact schema is open, but it must be explicit and machine-readable.

The backend must validate AI output before using it. If the model produces malformed output, the system should attempt a controlled repair or retry. Repeated failures must become an explicit operational failure rather than being silently accepted.

AI output should be traceable to the underlying conversation and retrieved protocol evidence.

16. Escalation Consensus and Clinical Safety

Escalation is the most safety-sensitive part of the system.

Do not rely entirely on a single model response to determine whether a patient needs human attention. The platform should use multiple independent assessments, agents, models, or equivalent reasoning paths and compare their results.

The exact consensus mechanism is open. You may use majority voting, weighted voting, an arbiter, rule-assisted consensus, or another defensible approach. What matters is that disagreement is detected and handled rather than hidden.

For example, if one assessment classifies a patient as routine while another identifies a protocol red flag, the system should recognize the disagreement and apply a conservative escalation strategy.

The system should record the individual assessments, disagreement, consensus result, evidence used, and final escalation decision.

Potential escalation situations include emergency symptoms, deterioration, severe or worsening symptoms, protocol-defined red flags, concerning patient statements, or significant uncertainty.

The AI must not diagnose the patient, prescribe medication, change treatment, or provide unsupported clinical judgment.

Patient-provided content, retrieved documents, and model-generated text must never be able to override system policies, authorization boundaries, or clinical safety rules. Treat these as separate trust domains.

17. Safety Evaluation and False-Negative Measurement

Create a fixed safety evaluation dataset containing known routine, concerning, urgent, ambiguous, incomplete-information, conflicting-information, and adversarial cases.

Run the triage and escalation system against this dataset and compare expected outcomes with actual outcomes.

At minimum, report true positives, false positives, true negatives, false negatives, and the **false-negative rate**. False negatives are particularly important because incorrectly failing to escalate a high-risk case can be more dangerous than producing an unnecessary escalation.

Safety evaluation should be repeatable after changes to models, prompts, retrieval, protocols, or consensus logic. The report should include representative disagreement cases, limitations, observed weaknesses, and improvements.

The evaluation must be reproducible and auditable rather than being a collection of manually selected examples.

18. Controlled AI Tools and Persistent Context

AI agents must interact with the application through controlled tools.

Possible tools include patient lookup, encounter lookup, protocol search, observation lookup, campaign status lookup, callback scheduling, call outcome recording, escalation creation, notification requests, communication recording, and mock EHR updates.

The general pattern should be:

AI request → **authorization** → **schema validation** → **business rules** → **execution** → **audit** → **result**

The AI must never receive unrestricted database access.

The platform should also maintain relevant patient outreach context across calls. This may include previous outcomes, callback preferences, symptoms already discussed, unanswered questions, escalation state, and outstanding follow-up requirements.

Context retrieval should be task-specific. The system should not blindly provide the entire patient history to every model call.

19. Documentation and Call Records

Every outreach attempt should produce a structured record.

A call record should identify the patient, hospital, campaign, call, timestamps, outcome, conversation information, AI assessments, triage result, escalation decision, documentation status, and relevant follow-up actions.

Supported outcomes should include successful completion, no answer, busy, voicemail, dropped call, invalid number, declined interaction, callback request, escalation, technical failure, and manual follow-up.

Documentation should be traceable back to the conversation. The system should make it possible for a reviewer to understand how a documented conclusion was produced.

Where applicable, the platform should send the resulting structured documentation to the mock EHR.

20. Workflows, Events and Notifications

The system should use asynchronous events and background processing for operations such as discharge ingestion, campaign preparation, queue scheduling, calls, retries, EHR synchronization, notifications, analytics, and evaluation.

Important events include patient import, campaign creation/start/pause, call scheduling/start/completion/failure, retry scheduling, callback requests, escalation creation/acknowledgement/resolution, EHR updates, and notification delivery.

Events may trigger operational workflows. For example, an escalation may notify a nurse and then notify a backup reviewer if it is not acknowledged within a configured period. A failed call may trigger retry eligibility and return the task to scheduling.

The workflow system should account for duplicate events, retries, failed consumers, and out-of-order processing. Important operations must be idempotent.

Where practical, non-engineering users should be able to configure operational behavior such as escalation notifications, reviewer timeouts, callback reminders, campaign pauses, and retry notifications without changing AI logic.

21. Escalation Management and Human-in-the-Loop

Escalations are first-class operational records.

An escalation should contain the patient, hospital, campaign, call, trigger, clinical indicators, triage result, consensus result, priority, creation time, assigned reviewer, status, resolution, and resolution timestamp.

A reasonable lifecycle is:

State	Meaning
OPEN	Created and awaiting handling
ASSIGNED	Assigned to a reviewer
IN_REVIEW	Human review underway
WAITING_FOR_INFORMATION	Additional information required
RESOLVED	Human action completed
CLOSED	Optional final state

Authorized staff should be able to inspect the patient context, conversation, AI outputs, protocol evidence, and escalation rationale before resolving the case.

Staff should be able to acknowledge, assign, reassign, resolve, and record follow-up actions.

Notifications may use an internal dashboard, email, SMS, or another configurable mechanism. Notification delivery should be observable.

Human actions, operational overrides, and clinical resolutions must be auditable.

22. Dashboards and Analytics

The platform should provide operational dashboards rather than only technical monitoring.

The **Campaign Manager dashboard** should show campaign progress, queue state, pending and active work, retries, completed and failed tasks, escalations, manual follow-ups, active call capacity, queue depth, oldest pending task, and patients approaching their clinical cutoff. Campaign managers should also be able to perform permitted controls such as starting, pausing, resuming, or reprioritizing campaigns.

The **Hospital Admin dashboard** should provide hospital-wide visibility into campaigns, outreach volume, contact rates, escalations, manual follow-ups, staff activity, EHR synchronization, protocols, and system health.

The **Platform Admin dashboard** should provide authorized aggregate visibility across hospitals, including hospital activity, campaign volume, queue health, AI usage, errors, escalation statistics, and system performance.

Authorized users should also have a patient operational view showing discharge information, outreach status, call history, outcomes, escalations, observations, documentation, and follow-up actions.

Analytics should cover campaign performance, including eligible patients, attempted and completed outreach, contact outcomes, average attempts, time to contact, queue wait, retry behavior, capacity utilization, escalations, and manual follow-ups.

Hospital and platform analytics may aggregate these metrics across authorized tenants, but aggregate analytics must not become a mechanism for bypassing patient-level access controls.

23. Observability, Audit and System Health

The system must make its behavior observable.

Application monitoring should cover API latency and errors, authentication failures, background jobs, queue depth, scheduling latency, concurrency utilization, retry backlog, stuck tasks, calling failures, notification failures, and EHR failures.

AI observability should include the agent or model used, latency, retrieval sources, structured output validation, tool usage, disagreement, escalation evidence, token usage, estimated cost, and failures where practical.

Sensitive healthcare information should not be unnecessarily written into logs.

Important operations should produce audit records, including campaign creation and pause actions, AI escalation assessments, consensus decisions, staff reviews, EHR updates, and operational overrides.

Audit records should not be casually editable or deletable.

The system should expose basic health states such as **Healthy, Degraded, and Unavailable**, along with queue-specific health such as active calls, capacity, pending work, oldest task, cutoff-risk tasks, failed tasks, and stuck workers.

Correlation or operation identifiers should be used where useful for tracing asynchronous workflows.

24. Reliability, Idempotency and Recovery

The platform should explicitly handle failures rather than assuming every external service succeeds.

Important failure cases include AI provider outages, voice provider failures, database failures, EHR failures, queue worker crashes, notification failures, malformed AI responses, timeouts, rate limits, and interrupted calls.

Use appropriate timeouts, retries, backoff, circuit breaking or fallback mechanisms, explicit failure states, and manual recovery paths.

Operations such as starting a call, recording an outcome, creating an escalation, sending a notification, writing to the EHR, scheduling a retry, processing a discharge, and consuming an event should be idempotent where duplicate execution could cause harm.

Stuck tasks should be recoverable through heartbeat or timeout mechanisms. A failed worker should not permanently consume outbound capacity.

The system should prevent duplicate calls, duplicate escalations, and duplicate external side effects.

25. Security, Privacy and Clinical Guardrails

The platform must demonstrate healthcare-data security awareness.

Required controls include authentication, authorization, RBAC, tenant isolation, secure APIs, input validation, secret management, audit logging, encryption where appropriate, data minimization, and protection against sensitive information appearing unnecessarily in logs.

Tenant context must be enforced consistently across APIs, databases, background workers, AI retrieval, tools, EHR integrations, and analytics.

AI-specific security must include protection against prompt injection and malicious patient or document content. Patient text and retrieved documents must not be allowed to override system instructions, authorization rules, or clinical safety policies.

The prototype should clearly distinguish between **prototype-level security and production compliance**. Do not claim HIPAA, SOC 2, or other certification unless the implementation actually satisfies the relevant requirements.

Document what would need to change before the system could be used with real patient data in production.

26. Architecture, APIs and Data Model

The exact technology stack and architecture are intentionally open.

Your architecture should clearly separate the major responsibilities: frontend and authentication, tenant management, campaign management, eligibility, queue and scheduling, outbound calling, AI agents, protocol retrieval, healthcare data and EHR integration, background processing, workflows and notifications, analytics, and observability.

Do not submit multiple repetitive architecture diagrams. One clear system architecture diagram is sufficient, supported by small state or sequence diagrams only where they materially improve understanding.

The important architectural boundaries are the interaction between AI and controlled tools, queue and calling infrastructure, the EHR abstraction, asynchronous events/workflows, and observability.

The API should support authentication, hospitals and users, patients and encounters, campaigns, queue state, calls, escalations, protocols and knowledge, EHR operations, notifications, analytics, evaluation, and health monitoring.

APIs should have validation, authorization, consistent responses, pagination where appropriate, error handling, and separation between transport logic and business logic.

The data model should capture relationships between hospitals, users, patients, encounters, discharges, campaigns, outreach tasks, calls, escalations, documentation, queue state, retries, events, audit records, and AI usage.

Design the schema with tenant isolation, indexing, history, operational state, and auditability in mind.

27. AI Model Abstraction and AI Usage

Avoid unnecessary coupling to a single AI provider.

Where practical, define interfaces for conversational generation, structured generation, classification, retrieval, evaluation, speech processing, and document understanding.

AI usage should be observable. Record the model or provider, agent, request purpose, latency, success/failure, token usage, and estimated cost where available.

Prompts should have clear responsibilities, expected inputs and outputs, versioning, and evaluation coverage. At minimum, distinguish prompts used for voice intake, clinical triage, escalation assessment, consensus, documentation, and evaluation.

The candidate should explain why particular models or AI approaches were selected and what tradeoffs were made.

28. Performance and Load Expectations

The prototype should be capable of handling hundreds of patient records, multiple hospitals and campaigns, concurrent outbound work, retries, background jobs, and dashboard updates.

You do not need to demonstrate production-scale infrastructure, but you should explain how the system could scale. Discuss worker scaling, queue coordination, database indexing, caching where useful, event processing, AI latency, and voice-provider limitations.

The required 20–30 patient queue simulation should demonstrate that the architecture continues to behave correctly when concurrency is intentionally constrained.

29. Testing and Evaluation

Testing should cover both normal functionality and failure conditions.

Backend testing should include authentication, authorization, tenant isolation, API validation, campaign lifecycle, and eligibility. Queue testing should cover prioritization, concurrency, retries, backoff, deadlines, callbacks, worker failure, recovery, and idempotency.

AI testing should cover structured output validation, protocol grounding, unsupported clinical information, triage behavior, escalation behavior, and disagreement handling.

Safety testing must include known escalation and non-escalation cases and must report false negatives.

Background processing should be tested for retries, duplicate events, failed workers, and recovery. EHR integration should be tested independently from the AI layer.

The fixed safety evaluation dataset should be repeatable so that model, prompt, retrieval, or protocol changes can be compared against previous results.

30. Technology Selection and Deployment

Candidates may choose their own frontend, backend, database, queue, AI models, retrieval system, voice infrastructure, hosting platform, and supporting technologies.

The important requirement is to justify the choices based on reliability, development speed, scalability, maintainability, safety, and prototype constraints.

The preferred outcome is a publicly accessible deployed prototype containing the frontend, backend, database, AI components, queue workers, dashboards, authentication, mock EHR, and background processing. Real telephony is encouraged but not mandatory.

Secrets must be supplied through environment or deployment configuration and must never be committed to source control.

If authentication is required for evaluation, provide safe demo credentials or another documented evaluation mechanism.

31. Prototype Scope and Prioritization

Because this is a **3–4 day prototype**, scope management is part of the evaluation. Candidates should prioritize the operational core instead of attempting to implement every optional feature.

Must Have

The prototype must demonstrate multi-tenancy, hospital onboarding, RBAC, tenant isolation, hospital configuration, patient and discharge ingestion, structured healthcare data, eligibility, campaigns, outreach state, patient history/timeline, and campaign monitoring.

The outbound queue is mandatory and must demonstrate centralized concurrency control, prioritization, retry and backoff, no-answer/busy/voicemail handling, dropped calls, callbacks, maximum retries, manual follow-up, clinical cutoffs, and failure recovery.

The AI layer must demonstrate voice intake or a convincing conversation simulation, structured clinical triage, hospital-grounded knowledge retrieval, multiple escalation assessments or equivalent consensus logic, structured documentation, and controlled AI tools.

The prototype must include a mock EHR, safety evaluation dataset, false-negative measurement, audit logging, dashboards, queue visibility, escalation handling, background processing, event workflows, observability, error handling, idempotency, tests, and deployment documentation.

Should Have

Strong enhancements include real outbound calling, streaming voice, realtime dashboards, advanced queue prioritization, patient callback preferences, configurable workflows, FHIR-shaped resources, improved retrieval, EHR synchronization, escalation notification chains, and advanced AI tracing, cost, and evaluation capabilities.

Nice to Have

Optional additions include multilingual conversations, sentiment or conversation-difficulty detection, intelligent calling windows, predictive contact optimization, staff balancing, SMS

fallback, patient education, additional integrations, advanced clinical dashboards, risk prediction, and queue simulation tooling.

Optional functionality should never come at the expense of queue correctness or patient safety.

32. End-to-End Success Criteria

The completed system should demonstrate an end-to-end scenario in which a hospital is created and configured, users are assigned, discharge patients are imported, a campaign is created, eligible patients are identified, tasks enter the queue, limited calling capacity is enforced, different call outcomes occur, retries and callbacks are scheduled, an AI conversation collects structured information, clinical triage evaluates the interaction, multiple assessments produce an escalation decision, an urgent case enters the human workflow, staff review the case, documentation is generated, the mock EHR is updated, and dashboards reflect the resulting activity.

The queue should demonstrate that capacity is respected, higher-priority patients can move ahead when justified, patients approaching their clinical cutoff receive appropriate attention, retries are handled correctly, dropped calls do not disappear, callback requests are honored, maximum retries create manual follow-up, failed workers release capacity, and duplicate calls are prevented.

The safety workflow should demonstrate protocol grounding, multiple assessments, disagreement detection, conservative escalation, structured evidence, false-negative measurement, and auditable decisions.

The engineering implementation should demonstrate full-stack development, multi-tenancy, healthcare data modeling, EHR abstraction, queue and distributed processing, AI agents, retrieval, controlled tools, structured outputs, human-in-the-loop workflows, observability, security, testing, deployment, and failure recovery.

33. Required Deliverables

Submit the following:

1. Deployed Application

Provide the deployed application URL and safe demo credentials or access instructions.

2. Source Repository

Provide a public GitHub repository containing source code, README, setup instructions, environment configuration example, database setup, mock EHR, queue simulation, tests, and deployment instructions.

3. Demo Video

Provide a short demonstration covering hospital onboarding, patient ingestion, campaign creation, queue behavior, constrained concurrency, retries, AI interaction, triage, escalation, human review, documentation, EHR update, analytics, and system health.

4. Architecture Documentation

Provide one clear architecture diagram and concise explanation of the major services and boundaries. Additional diagrams should only be included when they clarify an important state or interaction.

5. Queue Design Document

Document queue states, scheduling logic, priority calculation, concurrency control, retry strategy, callback handling, clinical deadlines, fairness, failure recovery, and manual follow-up.

6. Safety Evaluation Report

Provide the fixed evaluation dataset, expected and actual outcomes, false positives, false negatives, false-negative rate, disagreement examples, limitations, and improvements.

7. AI Usage Documentation

Document the models, agents, retrieval approach, prompts, structured schemas, tools, consensus mechanism, evaluation process, and failure handling.

8. Development AI Usage

Where AI coding or development tools were used, document the important prompts or workflows used during architecture, database, backend, frontend, queue, retry, voice, triage, consensus, EHR, testing, debugging, and documentation work.

9. Known Limitations and Tradeoffs

Clearly document what was simplified or omitted. Examples may include simulated telephony,

mock EHR, simplified healthcare resources, limited languages, simplified protocols, prototype-level security, limited scale, or reduced clinical coverage.

The evaluator should be able to distinguish intentional prototype tradeoffs from incomplete or broken functionality.

34. Grading Focus

This project is intentionally designed to distinguish between **building an AI agent** and **building an AI-powered operational system**.

The strongest submissions will demonstrate systems thinking around queue management, concurrency, distributed processing, retries, failure recovery, multi-tenancy, AI safety, healthcare integration, observability, human-in-the-loop workflows, and evaluation.

The primary grading areas are:

Priority	Evaluation Area
1 — Critical	Queue design, concurrency, retry logic, multi-tenancy, AI triage, escalation consensus, safety evaluation
2 — Important	Campaign management, dashboards, mock EHR, workflows, observability, reliability
3 — Enhancement	Real telephony, advanced analytics, richer UI, additional integrations

A visually impressive UI should not compensate for an unsafe or unreliable queue. Likewise, an impressive AI conversation should not compensate for poor tenant isolation, missing escalation controls, or lack of failure handling.

Candidates should explicitly document features they chose not to implement.

35. Final Product Expectation

The final product should feel like one connected healthcare operations system.

Hospitals provide discharge data. Campaigns determine who requires outreach. Eligibility identifies the patients. The queue decides who should be contacted next while respecting capacity, clinical deadlines, retries, and callbacks. The voice layer conducts the interaction. Clinical triage converts the conversation into structured information. Hospital knowledge grounds the AI. Multiple assessments reduce dependence on a single model. Consensus determines whether human attention is required. Escalations enter a human workflow. Documentation records what happened. The EHR maintains structured records. Background workflows handle retries and notifications. Dashboards show operational performance. Observability reveals system health. Safety evaluation measures whether the AI behaves appropriately. Multi-tenancy keeps every hospital isolated.

The core challenge is therefore **not to build an AI phone bot.**

The challenge is to build a **multi-tenant, queue-driven, observable, safety-conscious AI healthcare operations platform** that can reliably coordinate post-discharge outreach from patient eligibility through human resolution.